

Generalized linear model validation

Bert van der Veen

University of Bayreuth

The model

Writing the linear model:

$$y_i = \alpha + \mathbf{x}_i \beta + \epsilon_i \sim \mathcal{N}(0, \sigma^2) \quad (1)$$

Is the same as:

$$\mathbb{E}(y_i | \mathbf{x}_i) = \alpha + \mathbf{x}_i \beta \quad (2)$$

as long as $\mathbb{E}(\epsilon_i) = 0$.

Generalised linear model

$$\begin{aligned}
 g\{\mathbb{E}(y_i|x_i)\} &= \eta_i = \alpha + x_i\beta \\
 \mathbb{E}(y_i|x_i) &= g^{-1}(\eta_i) = g^{-1}(\alpha + x_i\beta)
 \end{aligned}
 \tag{3}$$

GLMs do not have a residual term

GLM Assumptions

- ▶ No outliers
- ▶ Independence
- ▶ Correct distribution
- ▶ Correct link function
- ▶ Correct variance function (implied by previous two)

Outline

- ▶ Residual plots for checking GLM assumptions
- ▶ Issues with residuals in GLMs
- ▶ Types of residuals
- ▶ Residual diagnostics
- ▶ Dharma/quantile residuals
- ▶ Prediction error

Modeling checking

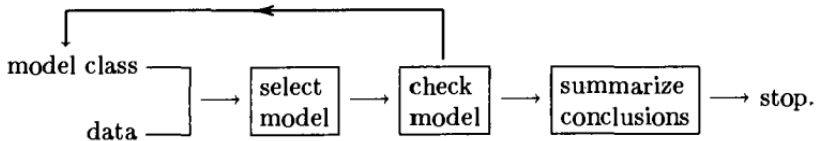


Figure 1: McCullagh and Nelder (1989) workflow

Methods for checking GLM assumptions

- ▶ Tests
- ▶ Outliers: Cook's distance, Residual vs. fitted
- ▶ Independence: Partial residual plots, Residuals vs. fitted, Residuals vs. lagged residuals
- ▶ Correct distribution: QQ-plot
- ▶ Correct link function: residuals vs. fitted, include η_i in the model (Hinkley 1985) , linear predictor against transformed response
- ▶ Correct variance function: Abs(residuals) vs. fitted

Response residuals

We could use the same residual as in linear regression:

$$\epsilon_i = y_i - \hat{\mu}_i$$

But we do not expect these to look nice in GLMs.

- ▶ Because the variance depends on the mean
- ▶ We want nice looking residuals when a model is good
- ▶ We want bad looking residuals otherwise

GLM residuals: Pearson's

Even though we lack the ϵ_i term in GLMs, we still calculate the residuals similarly

GLM residuals: Pearson's

Even though we lack the ϵ_i term in GLMs, we still calculate the residuals similarly

$$\epsilon_{i,pearson} = \frac{y_i - \hat{\mu}_i}{\sqrt{\text{var}(y_i; \hat{\mu}_i, \hat{\phi})}} \quad (4)$$

as the scaled difference between data and mean

GLM residuals: Pearson's

Even though we lack the ϵ_i term in GLMs, we still calculate the residuals similarly

$$\epsilon_{i,pearson} = \frac{y_i - \hat{\mu}_i}{\sqrt{\text{var}(y_i; \hat{\mu}_i, \hat{\phi})}} \quad (4)$$

as the scaled difference between data and mean

Approximately normally distributed in large samples

Recall deviance?

We do not have a ϵ_i term in GLMs, but we do have the deviance function

Recall deviance?

We do not have a ϵ_i term in GLMs, but we do have the deviance function

$$D(\mathbf{y}; \hat{\boldsymbol{\mu}}) = \sum_{i=1}^n 2[y_i \{g(y_i) - g(\hat{\mu}_i)\} - y_i + \hat{\mu}_i] \quad (5)$$

Which represents distance to the saturated model

GLM residuals: deviance

$$D(\mathbf{y}; \hat{\boldsymbol{\mu}}) = \sum_{i=1}^n 2[y_i \{g(y_i) - g(\hat{\mu}_i)\} - y_i + \hat{\mu}_i] \quad (6)$$

GLM residuals: deviance

$$D(\mathbf{y}; \hat{\boldsymbol{\mu}}) = \sum_{i=1}^n 2[y_i\{g(y_i) - g(\hat{\mu}_i)\} - y_i + \hat{\mu}_i] \quad (6)$$

- ▶ Note the summation over the observations
- ▶ Each observation has a defined contribution to the deviance
- ▶ $d_i = 2[y_i\{g(y_i) - g(\hat{\mu}_i)\} - y_i + \hat{\mu}_i]$
- ▶ We can use this to define a residual

GLM residuals: deviance

$$D(\mathbf{y}; \hat{\boldsymbol{\mu}}) = \sum_{i=1}^n 2[y_i\{g(y_i) - g(\hat{\mu}_i)\} - y_i + \hat{\mu}_i] \quad (6)$$

- ▶ Note the summation over the observations
- ▶ Each observation has a defined contribution to the deviance
- ▶ $d_i = 2[y_i\{g(y_i) - g(\hat{\mu}_i)\} - y_i + \hat{\mu}_i]$
- ▶ We can use this to define a residual

Deviance residuals

$$\epsilon_{i,deviance}^{\hat{}} = \text{sign}(y_i - \hat{\mu}_i) \sqrt{d_i}, \quad \text{so that } \sum_{i=1}^n \epsilon_{deviance,i}^{\hat{}}{}^2 = D(\mathbf{y}; \hat{\boldsymbol{\mu}}) \quad (7)$$

Approximately normally distributed in large samples

Why deviance residuals

- ▶ Converges faster to approximate normality
(Dunn and Smyth 2018, Cox and Snell 1968)
- ▶ Can be adjusted for small samples (Pierce and Schafer 1986)
 - ▶ e.g. Poisson $\mu_i^{-0.5}/6$

Why deviance residuals

- ▶ Converges faster to approximate normality
(Dunn and Smyth 2018, Cox and Snell 1968)
- ▶ Can be adjusted for small samples (Pierce and Schafer 1986)
 - ▶ e.g. Poisson $\mu_i^{-0.5}/6$

Still inappropriate for discrete data and small samples

In practice

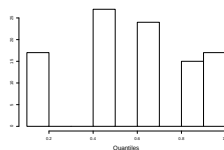
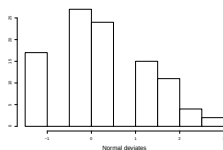
In practice, both Pearson's and Deviance residuals are often non-normally distributed.

Randomized Quantile residual (Dunn and Smyth 1996)

- ▶ Gold standard residual
- ▶ Better suited for small samples and discrete data types
- ▶ Exactly normally distributed
- ▶ Suitable for all kinds of models

Continuous

$$r_Q = \Phi^{-1} \left\{ \mathcal{F} \left(y_i; \hat{\mu}_i, \hat{\phi} \right) \right\} \quad (8)$$



Example: horseshoe crabs

Counts of male individuals (“satellites”) near females (Note: not a crab)



Figure 2: nwf.org



Figure 3: uwm.edu

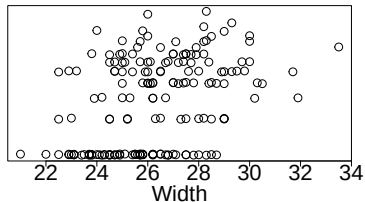
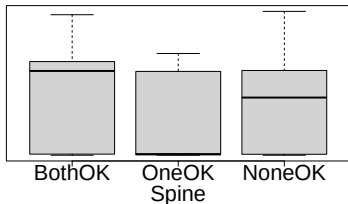
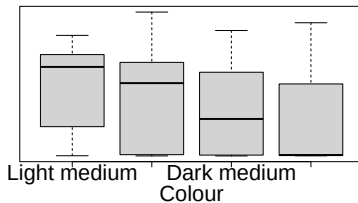
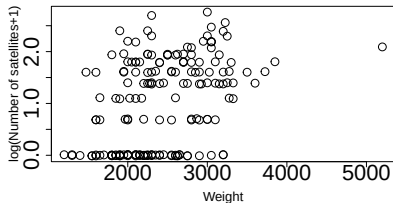
Horseshoe crabs: the data

Data originally from Brunswick (1996) via Agresti (2007) via Dunn and Smyth (2018)

- ▶ 173 observations
- ▶ 4 traits: colour, spine condition, width (cm), weight (g)

Col	Spine	Width	Sat	Wt
M	NoneOK	28.3	8	3050
DM	NoneOK	22.5	0	1550
LM	BothOK	26.0	9	2300
DM	NoneOK	24.8	0	2100
DM	NoneOK	26.0	4	2600
M	NoneOK	23.8	0	2100
LM	BothOK	26.5	0	2350

What is the relationship between the number of satellites and the female crabs' traits?



Horseshoe crabs: fit the model

```
model <- glm(Sat ~ Spine + Colour + Width + Weight,
              family = "poisson", data = hcrabs)
```


Horseshoe crabs: interpreting parameters

##		Estimate	Std. Error	z value	Pr(> z)
##	(Intercept)	-0.3600	0.97000	-0.37	0.7100
##	SpineOneOK	-0.1500	0.21000	-0.70	0.4800
##	SpineNoneOK	0.0870	0.12000	0.73	0.4700
##	ColourMedium	-0.2600	0.17000	-1.60	0.1200
##	ColourDark medium	-0.5100	0.20000	-2.60	0.0086
##	ColourDark	-0.5300	0.23000	-2.30	0.0190
##	Width	0.0170	0.04900	0.34	0.7300
##	Weight	0.0005	0.00017	3.00	0.0028

How do we interpret the intercept?

And its standard error?

Weight and Width uncentered

Horseshoe crabs: interpreting parameters

##		Estimate	Std. Error	z value	Pr(> z)
##	(Intercept)	1.3000	0.15000	8.70	4.4e-18
##	SpineOneOK	-0.1500	0.21000	-0.70	4.8e-01
##	SpineNoneOK	0.0870	0.12000	0.73	4.7e-01
##	ColourMedium	-0.2600	0.17000	-1.60	1.2e-01
##	ColourDark medium	-0.5100	0.20000	-2.60	8.6e-03
##	ColourDark	-0.5300	0.23000	-2.30	1.9e-02
##	WidthC	0.0170	0.04900	0.34	7.3e-01
##	WeightC	0.0005	0.00017	3.00	2.8e-03

Average weight: 2437.19g

Average width: 26.3cm

How do we interpret the intercept?

And its standard error?

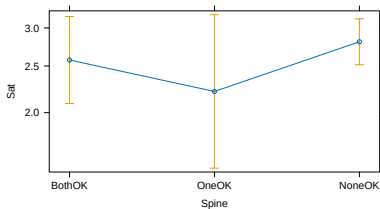
Weight and Width centered

Horseshoe crabs: interpreting parameters on the response scale

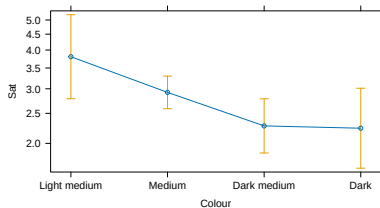
- ▶ (Intercept) = Light-medium coloured females with both spines in good condition, Weight $\approx 2437\text{g}$, Width $\approx 26.3\text{cm}$
- ▶ Width: $\exp(0.0170) = 1.017$, so double the number of satellites by $\log(2)/0.01709 = 40.56\text{cm}$ above average
- ▶ Weight: $\exp(0.0005) = 1.0005$, so double the number of satellites by $\log(2)/0.0005 = 1386.3\text{g}$ above average

Horseshoe crabs: visual interpretation

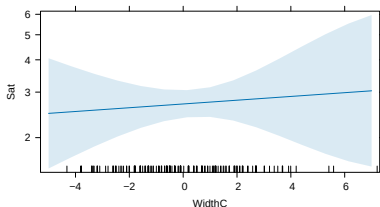
Spine effect plot



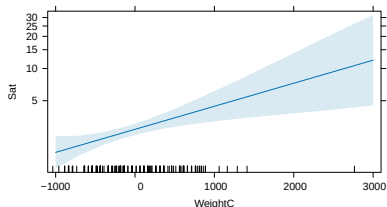
Colour effect plot



WidthC effect plot



WeightC effect plot



Horseshoe crabs: Negative-binomial

```
modelnb <- MASS::glm.nb(Sat ~ Spine + Colour + Width + Weight,
                        data = hcrabs)
```

and compare the models:

```
AIC(model, modelnb)
```

##	df	AIC
## model	8	920.8833
## modelnb	9	763.3204

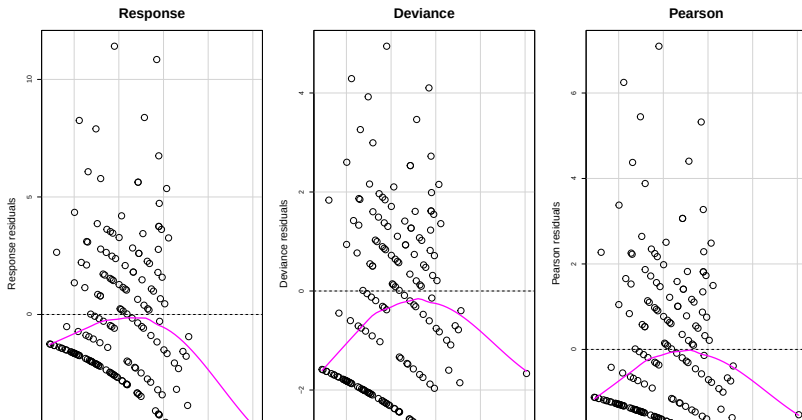
Horseshoe crabs: comparing estimates

##	Poisson estimate	NB estimate	Poisson SE	NB SE
## (Intercept)	1.3000	-0.2800	0.15000	2.00000
## SpineOneOK	-0.1500	-0.2400	0.21000	0.40000
## SpineNoneOK	0.0870	0.0430	0.12000	0.25000
## ColourMedium	-0.2600	-0.3200	0.17000	0.37000
## ColourDark medium	-0.5100	-0.6000	0.20000	0.42000
## ColourDark	-0.5300	-0.5800	0.23000	0.47000
## WidthC	0.0170	-0.0024	0.04900	0.10000
## WeightC	0.0005	0.0007	0.00017	0.00036

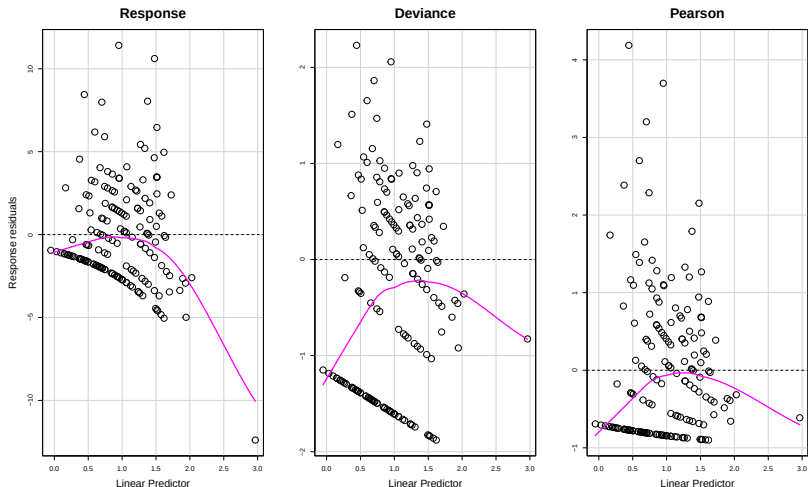
- ▶ SEs have doubled
- ▶ Coefficients have changed, but largely the same conclusions
- ▶ Except perhaps for the effect of “Width”

Horseshoe crabs: Poisson model residuals

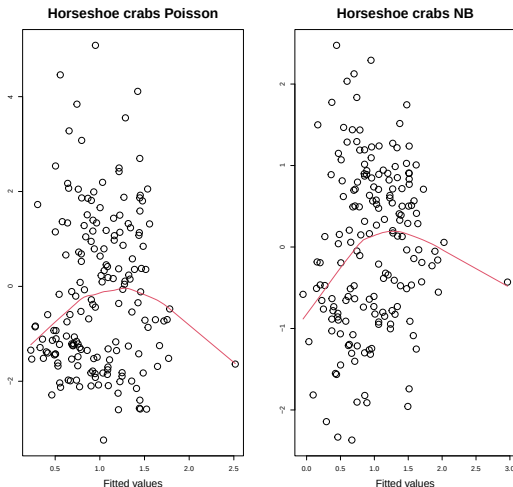
```
## Registered S3 method overwritten by 'car':
##   method      from
##   na.action.merMod lme4
```



Horseshoe crabs: NB model residuals



Horseshoe crabs: Dunn-Smyth residuals



Prediction

When you are only interested in predictions, e.g.:

- ▶ 20 years from now
- ▶ on a map
- ▶ new values of predictors x_i

Some assumptions might not matter. Generally, we still do not want structural deviations from the model.

Finding a model that predicts well

Usual procedure

1. Fit model to a part of the data ("train" and "test")
2. Predict for the remaining part ("test")
3. Quantify prediction error (e.g., RMSE)
4. Adjust model to minimize 3)

Notes

Finding a model that predicts well: better

unUsual procedure

1. Fit model to a part of the data ("~~train~~" and "~~test~~")
2. Predict for the remaining part ("~~test~~") Collect new data
3. Quantify prediction error (e.g., RMSE)
4. Adjust model to minimize 3)

Notes

Finding a model that predicts well: better

unUsual procedure

1. Fit model to a part of the data ("~~train~~" and "~~test~~")
2. Predict for the remaining part ("~~test~~") Collect new data
3. Quantify prediction error (e.g., RMSE)
4. Adjust model to minimize 3)

Notes

1. Statistically leaving out data is weird
2. We want as much information for our model as possible.
3. The model should be applicable beyond just the data that we observed

Quantifying prediction error

$$\text{MSE: } \frac{1}{n} \sum_{i=1}^n (y_i - \mu_i)^2$$

$$\text{RMSE: } \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \mu_i)^2}$$

$$\text{MAD: } \text{median}(|\mathbf{y} - \boldsymbol{\mu}|)^2$$

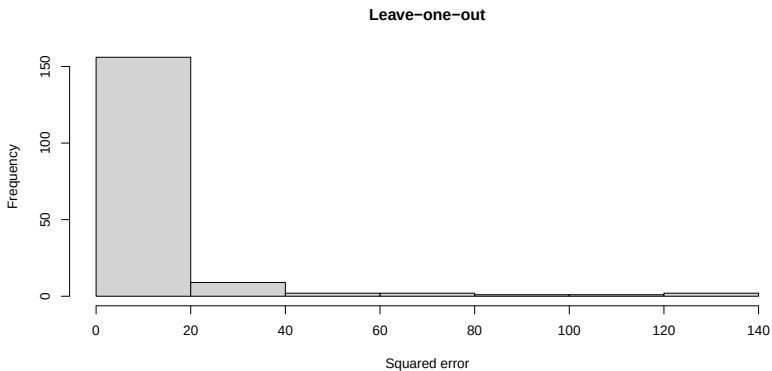
Or another metric. “AUC” is often used in binomial models.

For prediction, we want a model that performs well on one of these metrics.

Leave-one-out cross-validation Horseshoe crabs

```
SE <- NULL
for(i in 1:nrow(hcrabs)){
  set.seed(i)
  train <- hcrabs[-i,]
  model2 <- update(model2, data = train)
  SE <- c(SE, (hcrabs[i, "Sat"] -
    predict(model2, newdata = hcrabs[i, ], type="response"))^2)
}
```

Result



```
cat("RMSE: ", sqrt(mean(SE)))
```

```
## RMSE: 3.168937
```

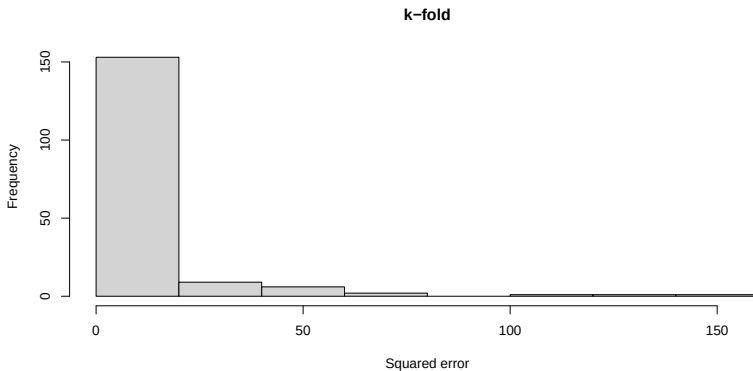

K-fold cross-validation Horseshoe crabs

```

k = 5
SE <- NULL
shuffle <- sample(1:nrow(hcrabs), nrow(hcrabs))
folds <- split(1:nrow(hcrabs), rep(1:k, length.out = nrow(hcrabs)))

for(i in 1:k){
  idx <- folds[[i]]
  test <- hcrabs[idx,]
  train <- hcrabs[-idx, ]
  model <- update(model2, data = train)
  SE <- c(SE, (test$Sat -
    predict(model, newdata = test, type="response"))^2)
}
  
```

Result



```
cat("RMSE: ", sqrt(mean(SE)))
```

```
## RMSE: 3.225183
```

Stratified cross-validation

- ▶ Cross-validation in a structured fashion: time or space.
- ▶ Select roughly equal partitions
 - ▶ Time: fit to a few years per partition
 - ▶ Space: areas
- ▶ I.e., blocking for non-independent responses

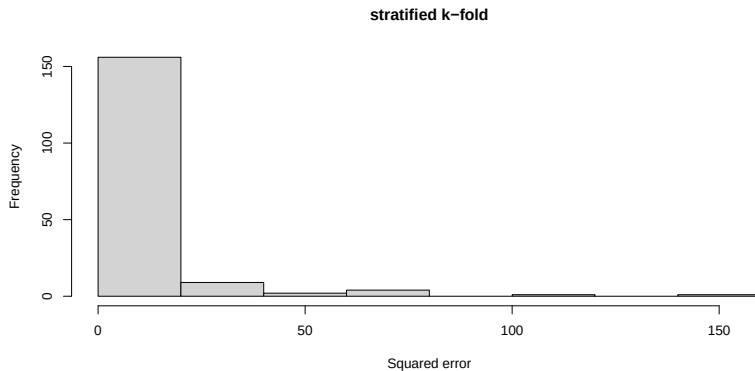
Stratified k-fold cross-validation Horseshoecrabs

Ensure a (roughly) similar distribution in every fold. For binary data, true stratified CV maintains the proportion.

```
k = 5
SE <- NULL
folds <- split(1:nrow(hcrabs), sample(rep(1:k, length.out = nrow(hcrabs))))

for(i in 1:k){
  test <- hcrabs[order(hcrabs$Sat),][folds[[i]],]
  train <- hcrabs[order(hcrabs$Sat),][-folds[[i]],]
  model <- update(model2, data = train)
  SE <- c(SE, (test$Sat -
predict(model, newdata = test, type="response"))^2)
}
```

Result



```
cat("RMSE: ", sqrt(mean(SE)))
```

```
## RMSE: 3.147592
```

Summary

- ▶ I did not mention Anscombe residuals, nor working residuals
- ▶ Do not draw conclusions before validating your model
- ▶ Use residual diagnostics/plots!
- ▶ If violated, adjust the model
- ▶ For prediction, look at cross-validation
 - ▶ Consider the structure of the data
 - ▶ Use independently collected data
 - ▶ There are many forms of cross-validation and quantifying error
 - ▶ Find model that minimizes the prediction error

Useful packages: car, DHARMA, boot